LAPORAN PRAKTIKUM
PROGRAM PENJUMLAHAN BCD

LAPORAN
Disusun untuk memenuhi tugas praktikum Mikroprosesor dan Interface

oleh :

Silfiana Dwi Nabiilah
221344028

Tanggal Percobaan: Jumat, 23 Februari 2024
Tanggal Penyerahan: Jumat, 01 Maret 2024

Instruktur: 1. Ferry Satria, BSEE., M.T.
2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI BANDUNG
2024

DAFTAR ISI

DAFTAR ISI 2
A. TUJUAN 3
B. LANDASAN TEORI 3
C. ALAT DAN BAHAN 6
D. LANGKAH KERJA 6
E. SOAL DAN HASIL 7
F. ANALISA DATA 10
G. KESIMPULAN 14
H. SARAN 14
DAFTAR PUSTAKA 15

A. TUJUAN

1. Memenuhi tugas mata kuliah Mikroprosesor dan Interface
2. Memahami prinsip kerja dari Aplikasi simulator EMU 8086
3. Membuat program sederhana menggunakan Bahasa Assembly
4. Menganalisa hasil program yang dibuat dengan Output yang dihasilkan mengenai penjumlahan heksadesimal

B. LANDASAN TEORI

1. Pengertian Mikroprosesor

Basis Data adalah sekumpulan informasi (data) yang berada di dalam komputer sehingga bisa diperiksa menggunakan sistem. Komponen dasar ini dijadikan sebagai sebuah sistem yang dapat digunakan untuk memudahkan pengguna dalam pengelolaan informasi yang dibutuhkan. Pada dunia pemrograman basis data sering kali digunakan untuk menyimpan konfigurasi, data pemograman, dan log hasil dari proses *running* aplikasi program.

Mikroprosesor merupakan CPU (Central Processing Unit) yang dikemas dalam satu buah IC (Integrated Circuit) atau chip. Mikroprosesor memiliki kemampuan untuk menerima input data digital kemudian diproses sesuai intruksi dalam memori untuk menghasilkan output. Mikroprosesor beroperasi dalam sistem angka biner dengan logika kombinasional dan logika digital sekuensial. Didalamnya terdapat

ALU (unit aritmatika dan logika), register, control unit, bus, dan clock untuk komputasi. Mikroprosesor mampu menjalankan perintah secara berurutan dalam bentuk program dengan intruksi yang menggunakan sistem bilangan biner. Akan tetapi, tidak semua mikroprosesor kompatibel untuk menjalankan berbagai program dalam kode intruksi yang berbeda. Beberapa jenis mikroprosesor seperti Z80 hanya dapat menjalankan intruksi untuk mikroprosesor 8080 atau 8085, sedangkan program yang menggunakan prosesor 8086 hanya dapat dijalankan dengan prosesor yang lebih tinggi seperti 80186, 80286, dan seterusnya. Sebaliknya, program 80186, 80286 dan seterusnya tidak berkemampuan untuk dijalankan dengan prosesor dibawahnya.

2. Pengertian EMU8086

Emu8086 adalah emulator mikroprosesor 8086 yang dapat digunakan untuk mengemulasikan perangkat keras pada komputer Windows. Emulator ini memungkinkan pengguna untuk menjalankan program yang dibuat untuk mikroprosesor 8086 pada komputer modern.

Emu8086 juga dilengkapi dengan assembler terintegrasi dan tutorial untuk pemula. Emulator ini dapat digunakan untuk mengeksplorasi dan mempelajari bahasa pemrograman assembly serta untuk mengembangkan dan menguji kode assembly pada mikroprosesor 8086.

3. Bahasa Assembly

Bahasa Assembly (rakitan) adalah bahasa pemrograman tingkat rendah yang digunakan pada perangkat yang dapat di program seperti komputer dan mikroprosesor. Instruksi-instruksi pada bahasa assembly menggantikan kode-kode biner dari bahasa mesin dengan *mnemonic* yang mudah diingat. Misal, instruksi penambahan dalam bahasa mesin memiliki kode “10110011” sedangkan dalam bahasa assembly dapat dibuat dalam instruksi mnemonic ADD, sehingga lebih mudah diapahami oleh manusia.

Bahasa assembly mempunyai hubungan 1-1 dengan berdasarkan instruksi mesin. Contohnya, untuk setiap julukan atau *mnemonic* yang nanti akan ditulis dengan menggunakan bahasa assembly, maka nanti juga akan langsung diterjemahkan dengan tepat satu kode operasi sehingga bisa langsung dimengerti oleh komputer. Untuk memahami cara kerja bahasa Assembly, Berikut ini merupakan tahapan-tahapan bagaimana sebuah huruf “A” bisa tampil di layar monitor menggunakan bahasa Assembly:

1. Microprocessor mengisi register di internal mikroprosesor dengan kode ASCII “A”, yaitu 41h.
2. Microprocessor mengirim sinyal interupt 21h. Akibatnya interupsi 21h dijalankan, yaitu mengirimkan “A” ke monitor.
3. Microprocessor mengirimkan sinyal interupt 20h tanda sudah selesai.

Dalam sebuah Mikroprocessor tempat menampung data-data yang akan diolah oleh prosesor. Register memiliki banyak jenis diantara lain :

1. Register Umum
   - Register AX (AH+AL) / *Accumulator Register*

     Register ini digunakan untuk menyimpan dan membaca data yang berhubungan dengan operasi aritmatika yang meliputi perkalian, pembagian, penjumlahan, dan pengurangan, operasi logika, dan pengiriman data.
   - Register BX (BH+BL) / *Base Register*

     Register ini digunakan untuk pengalamatan yang memungkinkan data diambil atau ditulisa langsung baik dari atau ke memori. Secara umum register ini digunakan sebagai lokasi untuk menyimpan alamat offset data dalam memori.
   - Register CX (CH+CL) / *Counter Register*

     Register ini digunakan untuk menghitung kode segment secara berulang menggunakan perintah loop.
   - Register DX (DH+DL) / *Data Register*

     Register DX adalah register serbaguna dengan beberapa fungsi utama yang berguna untuk mendukung register AX, termasuk dukungan untuk register AX ketika melakukan operasi aritmatika, terutama untuk operasi penggandaan dan pembagian 16 atau 32 bit data, dan Menyimpan data hexadesimal dalam Kode ASCII dalam register DL, yang ditampilkan pada layar monitor dan juga menyimpan dan menampilkan nomor port dalam mode port.
2. Register Segment
   - *Code Segment* (Register CS)

     Register yang digunakan untuk mencatat alamat segment dari kode program atau instruksi.
   - *Data Segment* (Register DS)

     Register yang digunakan untuk menampilkan alamata data dalam register transmisi.

- *Stack Segment* (Register SS)

  Register yang memiliki fungsi panggilan (CALL) dan meneruskan operasi stack ke program utama (RET).

- *Extra Segment* (Register ES)

  Register yang digunakan sebagai memori tambahan untuk operasi string pada CX dalam register tujuan umum.

3. Pointer Register

   - Register IP (*Instruction Pointer*)

     Register yang digunakan untuk menampilkan alamat perintah atau baris perintah dalam program.

   - Register SP (Stack *Pointer*)

     Register yang digunakan untuk menunjukkan byte terakhir dalam operasi susun.

   - Register BP (*Base Pointer*)

     Register yang memiliki fungsi kurang lebih sama dengan register BX, membaca dan menulis data langsung dari atau ke memori.

4. Flag Register

   Flag Register berfungsi sebagai penanda yang menunjukkan status atau keadaan dari suatu mikroprosessor. Bit-bit data pada flag yang berfungsi sebagai penanda status, akan mengalami perubahan tergantung kepada proses yang sedang berlangsung.

C. ALAT DAN BAHAN

1. Laptop atau PC
2. Simulator EMU 8086

D. LANGKAH KERJA

1. Mendownload aplikasi simulator EMU 8086 pada laptop atau PC.
2. Instalisasi aplikasi EMU 8086 pada laptop atau PC.
3. Memulai untuk membuat New Project pada aplikasi Emu 8086.
4. Memulai membuat pemrograman menggunakan Bahasa Assembly.
5. Menguji hasil pemrograman dengen mencompile pada aplikasi Emu8086.

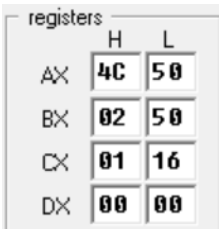
E. SOAL DAN HASIL

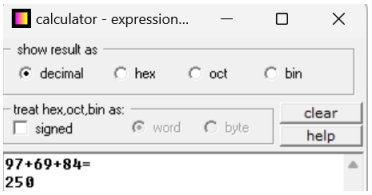
1) Buat program untuk menjumlahkan 3 data desimal berikut: 97 + 69 + 84. Simpan hasil penjumlahan pada register BX.

Jawab:

Program	Hasil	Kalkulator

```
01 TITLE LAB03 NO 1
02 .MODEL SMALL
03 .STACK 100H
04
05 .DATA
06 .CODE
07 MAIN PROC
08       MOV AH,0
09       MOV AL,97H
10       ADD AL,69H
11       DAA
12       ADC AH,0
13       ADD AL,84H
14       DAA
15       ADC AH,0
16       MOV BX,AX
17
18
19       MOV AH, 4CH
20       INT 21H
21
22 MAIN ENDP
23 END MAIN
```



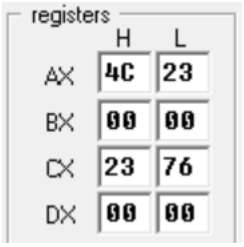


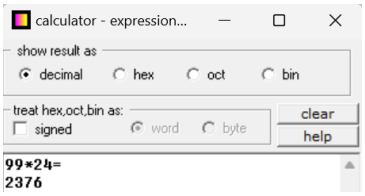
2) Buat program untuk menjumlahkan 24 data desimal bernilai 99. Simpan hasil penjumlahan pada register CX.

Jawab:

Program	Hasil	Kalkulator

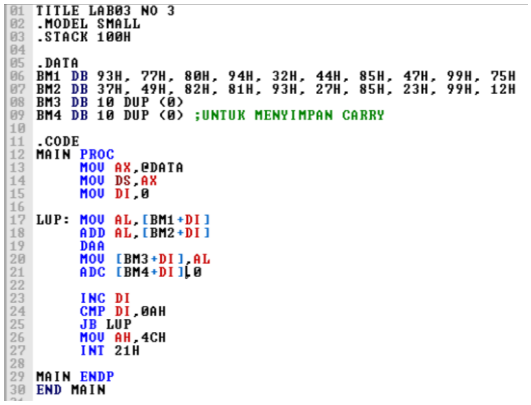
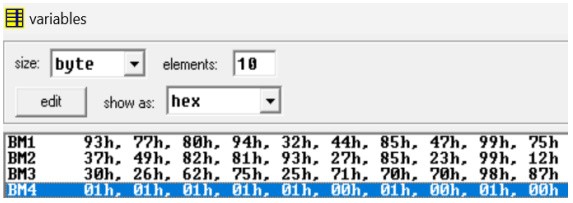
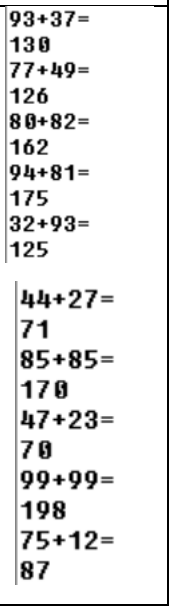
```
01 TITLE LAB03 NO 2
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05
06 .CODE
07 MAIN PROC
08       MOV CX,0
09       MOV AH,24
10
11 PROS:MOV AL,CL
12       ADD AL,99H
13       DAA
14       MOV CL,AL
15       MOV AL,CH
16       ADC AL,0
17       DAA
18       MOV CH,AL
19       DEC AH
20
21       JNZ PROS
22       MOV AH,4CH
23       INT 21H
24
25 MAIN ENDP
26 END MAIN
```





3) Buat diagram alir dan program untuk menjumlahkan 10 pasang data desimal 2 digit yang berada pada BM1 dan BM. Simpan hasil penjumlahan pada BM3 dan BM4 (untuk menyimpan carry).

Jawab:

Program	Hasil	Kalkulator
		

```
TITLE LAB03 NO 3
.MODEL SMALL
.STACK 100H

.DATA
BM1 DB 93H, 77H, 80H, 94H, 32H, 44H, 85H, 47H, 99H, 75H
BM2 DB 37H, 49H, 82H, 81H, 93H, 27H, 85H, 23H, 99H, 12H
BM3 DB 10 DUP (0)
BM4 DB 10 DUP (0) ;UNTUK MENYIMPAN CARRY

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV DI,0

LUP: MOV AL,[BM1+DI]
     ADD AL,[BM2+DI]
     DAA
     MOV [BM3+DI],AL
     ADC [BM4+DI],0

     INC DI
     CMP DI,0AH
     JB LUP
     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```

variables

size: byte elements: 10

edit show as: hex

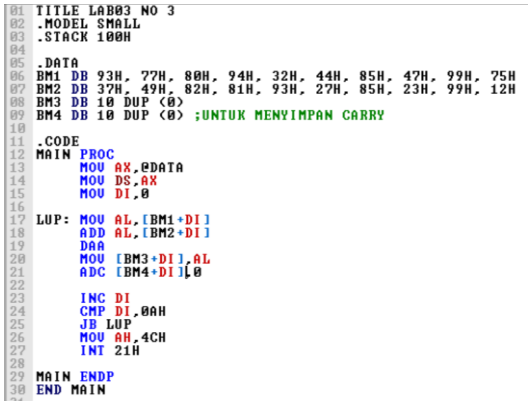
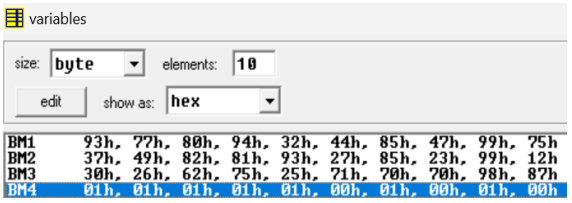
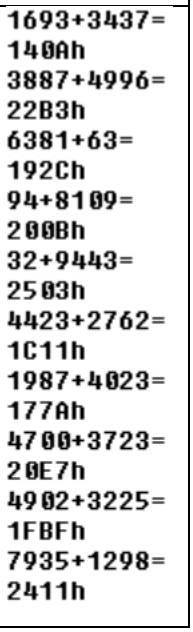
BM1	93h,	77h,	80h,	94h,	32h,	44h,	85h,	47h,	99h,	75h
BM2	37h,	49h,	82h,	81h,	93h,	27h,	85h,	23h,	99h,	12h
BM3	30h,	26h,	62h,	75h,	25h,	71h,	70h,	70h,	98h,	87h
BM4	01h,	01h,	01h,	01h,	01h,	00h,	01h,	00h,	01h,	00h

```
93+37=
130
77+49=
126
80+82=
162
94+81=
175
32+93=
125

44+27=
71
85+85=
170
47+23=
70
99+99=
198
75+12=
87
```

4) Buat diagram alir dan program untuk menjumlahkan 10 pasang data desimal 4 digit yang berada pada BM1dan BM2. Simpan hasil penjumlahan pada BM3.

Jawab:

Program	Hasil	Kalkulator
		

```
TITLE LAB03 NO 3
.MODEL SMALL
.STACK 100H

.DATA
BM1 DB 93H, 77H, 80H, 94H, 32H, 44H, 85H, 47H, 99H, 75H
BM2 DB 37H, 49H, 82H, 81H, 93H, 27H, 85H, 23H, 99H, 12H
BM3 DB 10 DUP (0)
BM4 DB 10 DUP (0) ;UNTUK MENYIMPAN CARRY

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV DI,0

LUP: MOV AL,[BM1+DI]
     ADD AL,[BM2+DI]
     DAA
     MOV [BM3+DI],AL
     ADC [BM4+DI],0

     INC DI
     CMP DI,0AH
     JB LUP
     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```

variables

size: byte elements: 10

edit show as: hex

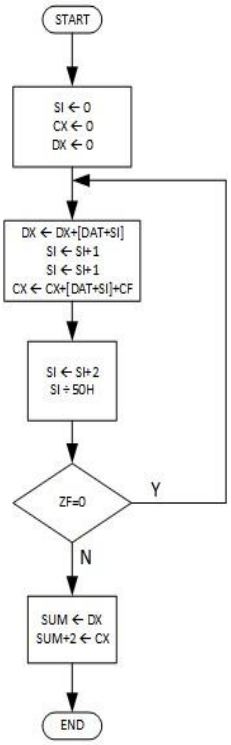
BM1	93h,	77h,	80h,	94h,	32h,	44h,	85h,	47h,	99h,	75h
BM2	37h,	49h,	82h,	81h,	93h,	27h,	85h,	23h,	99h,	12h
BM3	30h,	26h,	62h,	75h,	25h,	71h,	70h,	70h,	98h,	87h
BM4	01h,	01h,	01h,	01h,	01h,	00h,	01h,	00h,	01h,	00h

```
1693+3437=
140Ah
3887+4996=
22B3h
6381+63=
192Ch
94+8109=
200Bh
32+9443=
2503h
4423+2762=
1C11h
1987+4023=
177Ah
4700+3723=
20E7h
4902+3225=
1FBFh
7935+1298=
2411h
```

5) Buat diagram alir dan program untuk menjumlahkan data desimal 8 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3.

Jawab:

Diagram alir:



Program	Hasil

```
TITLE LAB02 NO5
.MODEL SMALL
.STACK 100H

.DATA
DAT DW 3949,0B8H, 1DA1H,0D4H, 6FB1H,49H, 20F6H,73H, 0BF31H,0DAH, 24F7H,54H, 9856H,63H, 0E11FH,0B4H, 6C03H,0D9H, 0B6ACH,0D8H
    DW 3949,0B8H, 1DA1H,0D4H, 6FB1H,49H, 20F6H,73H, 0BF31H,0DAH, 24F7H,54H, 9856H,63H, 0E11FH,0B4H, 6C03H,0D9H, 0B6ACH,0D8H
    DW 2 DUP(0)

.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX

     MOV SI,0
     MOV CX,0
     MOV DX,0

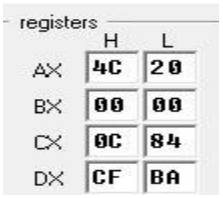
PROS: ADD DX,[DAT+SI]
      INC SI
      INC SI
      ADC CX,[DAT+SI]
      INC SI
      INC SI

      CMP SI,14H
      JNZ PROS

      MOV [SUM],DX
      MOV [SUM+2],CX

     MOV AH,4CH
     INT 21H

MAIN ENDP
END MAIN
```

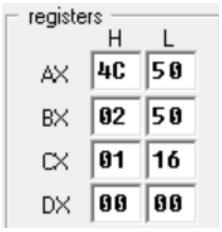


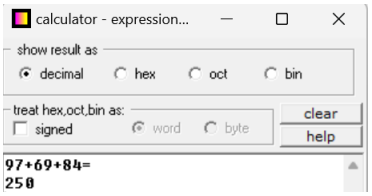
6) Buat diagram alir dan program untuk menjumlahkan 4 pasang data desimal 9 digit. Data input dan outputnya tentukan sendiri.

Jawab:

Program	Hasil	Kalkulator

```
01 TITLE LAB03 NO 1
02 .MODEL SMALL
03 .STACK 100H
04
05 .DATA
06 .CODE
07 MAIN PROC
08      MOV AH,0
09      MOV AL,97H
10      ADD AL,69H
11      DAA
12      ADC AH,0
13      ADD AL,84H
14      DAA
15      ADC AH,0
16      MOV BX,AX
17
18
19      MOV AH, 4CH
20      INT 21H
21
22 MAIN ENDP
23 END MAIN
```





F. ANALISA DATA

Pada pratikum kali ini dengan judul “Penjumlahan Data Heksadesimal” menggunakan aplikasi simulator EMU 8086. Untuk penulisan program, jika nilai angka dalam bentuk huruf yaitu lebih dari 9 digit maka harus diawali dengan angka 0, sedangkan jika angka berada pada rentang 0-9 maka tidak perlu diawali dengan angka 0.

Pada praktikum minggu ini penulis mempelajari tentang meteri “*looping*” menggunakan mnemonic jump. Mnemonic tersebut digunakan untuk memberikan perintah dengan tujuan mengarahkan proses pertama ke arah proses yang lainnya sesuai dengan label yang dibuat. Terdapat syarat tertentu sesuai dengan jenisnya. Pada praktikum ini menmonic jump yang digunakan berjenis JNZ (Jump Not Zero) yang berfungsi untuk melakukan loncatan ke Alamat tertentu dan akan bekerja bila flagnya bernilai 0. Terdapat banyak jenis mnemonic jump diantara lain: JMP, JE, JNE, JG, JL, JA, JB, JAE, dan JBE.

Agar proses penjumlahan dilakukan secara berurutan dari data pertama ke data selanjutnya, maka digunakanlah mnemonic INC (Increment) dimana operand akan dijumlahkan dengan nilai 1, operand yang dimaksud sejumlah dua digit data, sehingga bila menggunakan tipe data DW perlu dilakukan INC sebanyak 2 kali agar dapat menjumlahkan data yang berjumlah maksimal 4 buah data. Lalu untuk menentukan jumlah perulangannya penulis menggunakan mnemonic CMP (Compare), dimana dengan menggunakan mnemonic ini penulis dapat membatasi perulangan yang akan dilakukan oleh JNZ nantinya.

Untuk contoh dari *looping* sederhana dapat dilihat pada program no 1 sampai dengan no 9, semuanya merupakan contoh program *looping* yang hampir serupa dimana program menjalankan operasi penjumlahan dengan setiap data yang ada secara berurutan (data1,data2, data3,...).

Untuk menyimpan hasil dari operasi penjumlahan kedalam suatu unit memori, maka diperlukan satu buah variabel baru untuk mendeklarasaikan suatu register yang digunakan agar dapat menyimpan hasil/output. Dengan begitu setelah membuat sebuah proses untuk melakukan operasi perhitungan diperlukan menambahkan mnemonic MOV, contoh penulisannya “MOV HASIL,DX” dimana DX merupakan register tempat menyimpan hasil akhir, seperti yang kita tahu bahwasannya data yang berada di dalam DX diinputkan kedalam variabel HASIL.

Berikut merupakan detail dari perintah yang digunakan:

No	Perintah	Fungsi
1	TITLE	Perintah ini digunakan untuk memberikan judul pada program.
2	.MODEL SMALL	Perintah ini digunakan untuk menentukan model memori kecil.
3	.STACK 100H	Perintah ini digunakan untuk menetapkan ukuran stack sebesar 100H (256 bytes).
4	.DATA	Perintah ini digunakan untuk mendefinisikan dan menginisialisasi variabel data.
5	.CODE	Perintah ini digunakan untuk mendefinisikan dan menulis kode program.
6	.MAIN PROC	Perintah ini digunakan untuk memulai definisi prosedur utama.
7	.MOV AX,@DATA	Perintah ini digunakan untuk memuat alamat segmen data ke dalam register AX.
8	.MOV DS,AX	Perintah ini digunakan untuk menetapkan nilai register AX ke dalam register DS untuk mengatur segmen data.
9	MOV SI,0	Perintah ini digunakan untuk mengatur nilai awal SI (Source Index) ke 0.
10	MOV DX,0	Perintah ini digunakan untuk mengatur nilai awal DX (Destination Index) ke 0.
11	PENGULANGAN1	Perintah ini digunakan sebagai label untuk awal loop pertama
12	ADD DL,[BM1+SI]	Perintah ini digunakan untuk menambah nilai DL dengan nilai yang terdapat di lokasi memori [BM1+SI]
13	INC SI	Perintah ini digunakan sebagai increment SI untuk mengakses elemen berikutnya di dalam array.
14	ADC DH,0	Perintah ini digunakan untuk menambah nilai DH dengan carry flag, diatur ke 0 pada awalnya.

15	CMP SI,8H	Perintah ini digunakan untuk membandingkan nilai SI dengan 8 (panjang array). Jika belum mencapai 8, maka loncat ke PENGULANGAN1
16	JNZ PENGULANGAN1	Perintah ini digunakan untuk Jump jika nilai ZF (Zero Flag) tidak diatur, kembali ke PENGULANGAN1
17	MOV AH,4CH	Perintah ini digunakan untuk menetapkan nilai 4CH ke dalam register AH, menunjukkan fungsi untuk keluar dari program.
18	INT 21H	Perintah ini menggunakan interrupt 21H (INT 21H) untuk keluar dari program.
19	MAIN ENDP	Perintah ini digunakan untuk mengakhiri definisi prosedur utama.
20	END MAIN	Perintah ini digunakan untuk menandakan akhir dari program.

G. KESIMPULAN

Praktikum dengan judul “Penjumlahan Dat Heksadesimal” menghasilkan kesimpulan sebagai berikut :

1. Program berikut berfungsi untuk mengawali program:

   ```
   TITLE
   .MODEL SMALL
   .STACK 100H
   .DATA
   .CODE
   MAIN    PROC
   ```

2. Program berikut berfungsi untuk mengakhiri program:

   ```
   MOV AH,4CH
   INT 21H
   MAIN ENDP
   END MAIN
   ```

3. Pada perintah penjumlahan dengan 8 bit maka penjumlahan dilakukan dalam register low dengan carry yang diinputkan pada register high. Dalam prosesnya pada penjumlahan dengan 8 bit ini hanya dapat menginputkan 2 nilai dalam data untuk dijumlahkan secara berurutan dimulai dari yang paling rendah (kanan) lalu yang ke paling tinggi (kiri).
4. Instruksi ADD berfungsi untuk menjumlahkan data dengan maksimal 2 nilai atau 2 bit data, sedangkan jika dirasa hasil penjumlahannya menghasilkan nilai carry maka diperlukan untuk menggunakan instruksi ADC.
5. Setiap register memiliki 2 bagian yaitu bagial Low (AL) dan bagian High (AH). Kedua bagian ini dapat disatukan dalam 1 wadah yaitu AX
6. Perintah CMP (Compare) berfungsi untuk membandingkan dua nilai dan mengatur status flag (bendera status) berdasarkan hasil perbandingan.
7. Perintah JNZ (Jump Not Zero) berfungsi untuk melakukan lompatan (jump) ke alamat tertentu jika Zero Flag (ZF) tidak diatur (bernilai 0).
8. Perintah INC (Increment) berfungsi untuk menambah nilai suatu operand dengan 1. Operand tersebut bisa berupa register atau lokasi memori.

H. SARAN

Saran saya dalam melakukan praktikum ini yaitu ketelitian saat sedang menuliskan program harus ditingkatkan, karena tidak menutup kemungkinan terdapat kesalahan jika penulisan program kurang tepat.

DAFTAR PUSTAKA

Kho, D. “Pengertian Mikroprosesor dan Cara Kerja Mikroprosesor”. (Diakses pada 21 Februari 2024). Available : https://teknikelektronika.com/pengertian-mikroprosesor-microprocessor-cara-kerjamikroprosesor-adalah/

Catur. 2018. “Pengertian Register dan Jenis-Jenis Register”. (Diakses pada 21 Februari 2024). Available : http://garudacyber.co.id/artikel/1486-pengertian-register-dan-jenis-jenis-register.

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung.